ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 04

Primeros pasos con PHP

- Qué es PHP y cómo funciona una petición
- Vuestro primer ejemplo: phpinfo()
- Vuestro primer script: hola.php
- Sintaxis básica
- Variables y tipos de datos
- Operadores
- Condicionales
- switch y match
- Bucles
- Llaves o sintaxis alternativa
- Arrays
- Funciones
- Formularios: GET y POST
- Errores frecuentes
- Conectar con MySQL
- Glosario



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

¡Por fin! Tenemos el servidor, tenemos el editor... y hoy escribimos nuestro primer código. 🎉

Este bloque nos va a llevar **unas tres clases**. Os aviso de algo importante desde el principio: **son ejemplos de introducción**. Quiero que veáis cómo funciona la programación web con PHP, que escribáis código, que lo probéis y que perdáis el miedo. No vamos a profundizar en cada tema: en las clases siguientes volveremos sobre todo esto con calma y en detalle.

Así que no os agobiéis si algo no queda claro del todo. Ahora toca **probar, romper y volver a probar**.

•

📘 Cómo vamos a trabajar



1

🔍 Qué es PHP y cómo funciona una petición



2

⚠️ Vuestro primer ejemplo: phpinfo()



3

🚀 Vuestro primer script: hola.php



4

🤔 Sintaxis básica



5

👍 Variables y tipos de datos



6

📘 Operadores



7

🤔 Condicionales



8

👌 switch y match



9

🚀 Bucles



10

🔍 Llaves o sintaxis alternativa



11

📘 Arrays



12

👍 Funciones



13

⚠️ Formularios: GET y POST



14

🐞 Errores frecuentes



15
😮 Conectar con MySQL



16
🚀 Ejercicios propuestos



17
👋 Glosario





CLASE 04

📘 Cómo vamos a trabajar

Durante estas tres clases trabajaremos en **un único proyecto** llamado `php-basico`:

1. Pulsad **Iniciar todo** en Laragon.
2. Pulsad **Raíz** (se abre `C:\laragon\www`) y cread la carpeta `php-basico`.
3. Abrid esa carpeta en VS Code con **Archivo → Abrir carpeta**.
4. Pulsad **Recargar** en Laragon para que cree el virtual host.
5. Cada archivo que creéis se abrirá en `http://php-basico.test/nombre-del-archivo.php`.

Para cada tema seguiremos siempre el mismo método:

Paso	Qué hacer
01 📖	Leed la explicación
02 ⌨️	**Escribid** el ejemplo vosotros (nada de copiar y pegar) y probadlo
03 ✅	Comprobad el resultado con las pruebas que os propongo



📁 En la carpeta `ejemplos/` de este repositorio tenéis todos los ejemplos ya escritos, numerados en orden. Usadlos para **comparar** cuando algo no os funcione, no para saltaros el paso de escribirlos.

🗓️ Clase 4



CLASE 04 · PUNTO 1

🔍 Qué es PHP y cómo funciona una petición

PHP es un lenguaje de programación que se ejecuta **en el servidor** y genera páginas web dinámicas: su contenido cambia según el usuario, la hora o los datos guardados. Lo usan WordPress, Moodle y frameworks como Laravel [1].



¿Qué pasa cuando abrís `http://php-basico.test/hola.php`?

Esquema

```
Navegador  ──1──►  Apache  ──2──►  PHP  ◄──3──►  MySQL
(pide hola.php)    (servidor web)  (ejecuta      (datos, si
                                    el código)    hacen falta)
      ◄──────────────── 4 · Apache devuelve SOLO HTML ────────────────
```

El código PHP **nunca sale del servidor**. Al navegador solo le llega el HTML que PHP ha generado.

PHP frente a JavaScript

	⚡ JavaScript (navegador)	🐘 PHP
Dónde se ejecuta	En el ordenador del usuario	En el servidor
Quién ve el código	Cualquiera, con «Ver código fuente»	Nadie: solo llega el HTML
Para qué se usa	Interactividad de la página	Formularios, bases de datos, usuarios
Extensión del archivo	`.js`	`.php`



⚠️ **Fijaos:** un archivo `.php` **no funciona con doble clic**. Necesita un servidor web con PHP, y eso es exactamente lo que nos da Laragon.



CLASE 04 · PUNTO 2

⚠️ Vuestro primer ejemplo: phpinfo()

Antes de escribir código de verdad, vamos a comprobar **qué características tiene nuestro servidor**. PHP incluye para ello la función `phpinfo()`, que muestra su configuración completa [2].

🎯 **Objetivo:** ver la configuración de PHP en `http://php-basico.test/info.php`.

01-info.php — php-paso-a-paso/clase04 — Visual Studio Code

🐘 01-info.php

```php
<?php
phpinfo();
```

📄 Ejemplo: `01-info.php`

Hacedlo en este orden:

1. En VS Code, dentro de `php-basico`, cread el archivo `info.php` con el código de arriba.
2. Guardadlo.
3. Abrid `http://php-basico.test/info.php` en el navegador.

¿Para qué sirve phpinfo()?

`phpinfo()` **no crea páginas para vuestros usuarios**: sirve para **comprobar las características del servidor**. Cuando algo no funciona, es lo primero que se mira.

Dato	Qué os dice
PHP Version	La versión de PHP que está en uso (la que elegisteis en Laragon)
Server API	Cómo ejecuta Apache a PHP: «Apache 2.0 Handler» (módulo, PHP *Thread Safe*) o «CGI/FastCGI» (*Non Thread Safe*)
Loaded Configuration File	El `php.ini` que está leyendo PHP. Ahí se activan extensiones y se configuran los errores
display_errors	Si PHP muestra los errores en pantalla. Buscadlo con `Ctrl + F`
pdo_mysql y **mysqli**	Extensiones para conectar con MySQL. Las usaremos en el punto 15

✅ Dos pruebas para comprobarlo

Haced esto	Debéis obtener
Abrid `http://php-basico.test/info.php`	Una página con la cabecera «PHP Version» y vuestra versión
Pulsad `Ctrl + F` y buscad `pdo_mysql`	Una sección con «PDO Driver for MySQL: enabled»



🔐 **Fijaos:** `phpinfo()` revela datos delicados del servidor. Usadlo solo en vuestro equipo y **borrad** `info.php` antes de subir una web a internet.



CLASE 04 · PUNTO 3

🚀 Vuestro primer script: hola.php

Ahora sí, vuestro primer script de verdad. Un clásico. 😄

🎯 **Objetivo:** ejecutar vuestro primer script en `http://php-basico.test/hola.php`.

02-hola.php — php-paso-a-paso/clase04 — Visual Studio Code

02-hola.php

```php
<?php
echo "¡Hola, mundo!";
```

📄 Ejemplo: `02-hola.php`

1. En `php-basico`, cread `hola.php` con el código de arriba.
2. Guardad. No hace falta pulsar Recargar: el virtual host ya existe.
3. Abrid `http://php-basico.test/hola.php`.

✅ Dos pruebas para comprobarlo

Haced esto	Debéis obtener
Abrid `http://php-basico.test/hola.php`	El texto ¡Hola, mundo!
Clic derecho en la página → **Ver código fuente**	Solo el texto: ni rastro de `<?php`



⚠️ Si en la barra de direcciones pone `file:///`, habéis abierto el archivo con doble clic y PHP no se ejecuta. Debe empezar por `http://`.



CLASE 04 · PUNTO 4

🤔 Sintaxis básica

El código PHP va entre `<?php` y `?>`, y **cada instrucción termina en punto y coma**. Lo que queda fuera de esas etiquetas se envía tal cual al navegador.

Mezclar PHP y HTML

03-fecha.php — php-paso-a-paso/clase04 — Visual Studio Code

03-fecha.php

```php
<!DOCTYPE html>
<html lang="es">
<body>
    <h1>Mi primera página PHP</h1>
    <p>Hoy es <?php echo date("d/m/Y"); ?></p>
</body>
</html>
```

📄 Ejemplo: `03-fecha.php`

Mostrar información y comentar

04-echo-comentarios.php — php-paso-a-paso/clase04 — Visual Studio Code

04-echo-comentarios.php

```php
<?php
echo "Hola";                      // la forma más usada
echo "Hola", " ", "mundo";        // admite varios valores
echo "<strong>Negrita</strong>";  // también genera HTML
/* Comentario de
   varias líneas */
```

📄 Ejemplo: `04-echo-comentarios.php`

El echo corto: `<?=`

Cuando mezcláis PHP y HTML, vais a escribir muchísimas veces `<?php echo ...; ?>` solo para mostrar un valor. Para eso PHP tiene un atajo: el **echo corto** (también llamado echo contraído), `<?=`.

Estas dos líneas hacen exactamente lo mismo:

03-fecha.php — php-paso-a-paso/clase04 — Visual Studio Code

03-fecha.php

```php
<p>Hoy es <?php echo date("d/m/Y"); ?></p>
<p>Hoy es <?= date("d/m/Y") ?></p>
```

Fijaos en lo que desaparece: la palabra `echo` y el punto y coma final. `<?=` ya significa «abre PHP y muestra esto», y el `?>` de cierre hace de punto y coma.

Forma larga	Echo corto
`<?php echo $nombre; ?>`	`<?= $nombre ?>`
`<?php echo date("Y"); ?>`	`<?= date("Y") ?>`
`<?php echo "Hola, $nombre"; ?>`	`<?= "Hola, $nombre" ?>`

¿Cuándo usar cada uno?

- Para **mostrar un único valor dentro del HTML**, el echo corto: queda más limpio y se lee mejor.
- Para **varias instrucciones** (asignar variables, `if`, bucles…), la etiqueta normal `<?php ... ?>`. Dentro de `<?=` solo cabe una expresión que mostrar.



💡 El echo corto **siempre está disponible** desde PHP 5.4 y no depende de ninguna configuración [8]. No lo confundáis con la etiqueta corta `<?` a secas (sin `php` ni `=`): esa sí depende de la opción `short_open_tag` y os recomiendo no usarla nunca.



⚠️ **Curiosidad que os puede costar un rato:** PHP busca sus etiquetas en todo el archivo, **también dentro de los comentarios HTML** (`<!-- -->`). Si escribís `<?=` dentro de un comentario HTML, PHP intentará ejecutarlo y os dará un *Parse error*.

📄 Ejemplo: `03-fecha.php` (incluye las dos formas)

Lo volveréis a ver en las plantillas con formularios y en el ejemplo completo con MySQL, y más adelante en Laravel, cuyas vistas Blade usan una idea parecida con `{{ }}`.

Reglas que conviene recordar

Regla	Ejemplo
Cada instrucción termina en `;`	`echo "Hola";`
Las variables distinguen mayúsculas	`$nombre` y `$Nombre` son distintas
Si el archivo es solo PHP, omitid el `?>` final	Evita espacios en blanco accidentales



😅 **Fijaos:** olvidar el `;` es el error número uno al empezar. PHP señala la línea **siguiente** a la que tiene el fallo.



CLASE 04 · PUNTO 5

👍 Variables y tipos de datos

Una variable es un espacio con nombre donde guardáis un dato. En PHP **siempre empieza por** `$` y no hay que declarar su tipo: PHP lo deduce del valor.

05-variables.php — php-paso-a-paso/clase04 — Visual Studio Code

05-variables.php

```php
<?php
$nombre = "Lucía";
$edad = 25;
echo "Me llamo $nombre y tengo $edad años.";
```

📄 Ejemplo: `05-variables.php`

Cómo nombrar variables

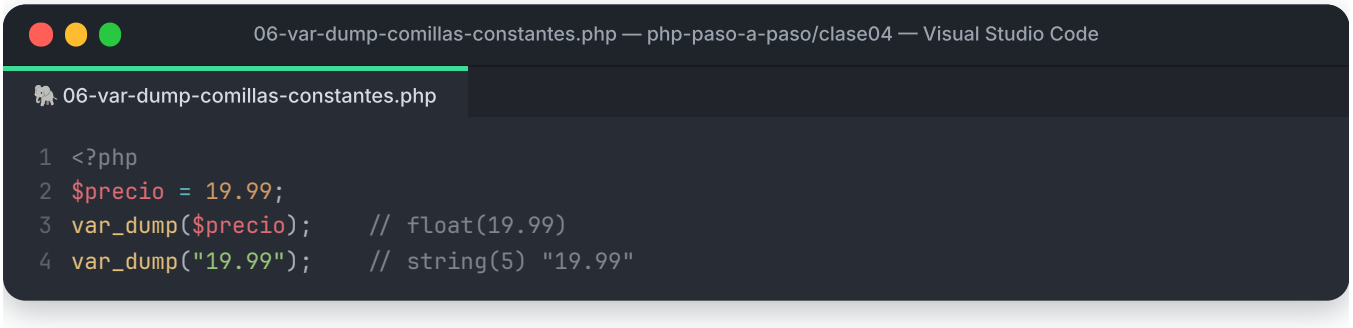
Correcto ✅	Incorrecto ❌	Por qué
`$edad`	`edad`	Falta el `$`
`$dato1`	`$1dato`	No puede empezar por número
`$precioTotal`	`$precio total`	Sin espacios: usad *camelCase*
`$anio`	`$año`	Evitad tildes y eñes

Tipos de datos básicos

Tipo	Qué guarda	Ejemplo
`int`	Números enteros	`$edad = 25;`
`float`	Decimales (con punto, no coma)	`$precio = 19.99;`
`string`	Texto entre comillas	`$ciudad = "Santander";`
`bool`	Verdadero o falso	`$activo = true;`
`array`	Varios valores	`$notas = [7, 8, 9];`
`null`	Sin valor	`$resultado = null;`

var_dump(), comillas y constantes

Mientras programáis, `var_dump()` muestra el **tipo** y el **valor** de una variable. Va a ser vuestra mejor herramienta para encontrar errores.



Con **comillas dobles** PHP sustituye las variables; con **comillas simples**, no:

06-var-dump-comillas-constantes.php — php-paso-a-paso/clase04 — Visual Studio Code

06-var-dump-comillas-constantes.php

```php
<?php
$nombre = "Ana";
echo "Hola $nombre";    // Hola Ana      → sustituye la variable
echo 'Hola $nombre';    // Hola $nombre  → texto literal
```

Las **constantes** guardan un valor que **no cambia** durante la ejecución. No llevan `$` y, por convenio, se escriben en mayúsculas:

06-var-dump-comillas-constantes.php — php-paso-a-paso/clase04 — Visual Studio Code

06-var-dump-comillas-constantes.php

```php
<?php
const IVA = 0.21;
define("NOMBRE_WEB", "Mi tienda");

$precio = 100;
echo "Precio con IVA: " . $precio * (1 + IVA);    // 121
```

📄 Ejemplo: `06-var-dump-comillas-constantes.php`

`const` o `define()`: ¿cuál uso?

Las dos crean constantes y las dos se usan igual (`IVA`, `NOMBRE_WEB`), pero no son lo mismo [9]:

- `const` es una palabra del lenguaje, como `if` o `echo`. PHP la lee **antes** de ejecutar nada, así que tiene que ir «suelta» en el código, en el nivel principal del archivo.
- `define()` es una **función**, así que se ejecuta cuando le llega el turno, como cualquier otra instrucción. Por eso puede ir dentro de un `if`, y su nombre y su valor se pueden calcular.

07-const-o-define.php — php-paso-a-paso/clase04 — Visual Studio Code

07-const-o-define.php

```php
<?php
const IVA = 0.21;
const PRECIO_ENVIO = 4.95;
const TOTAL_MINIMO = PRECIO_ENVIO * 10;   // const puede usar otras constantes

define("VERSION", "1.0");

// Dentro de un if solo vale define(): con const, PHP da un error de sintaxis
if (date("G") < 14) {
    define("SALUDO", "Buenos días");
} else {
    define("SALUDO", "Buenas tardes");
}

define("ANIO_ACTUAL", (int) date("Y"));   // un valor que sale de una función

$idioma = "ES";
define("MONEDA_" . $idioma, "€");          // un nombre construido: MONEDA_ES

echo SALUDO . " y bienvenidos (versión " . VERSION . ")";

if (defined("IVA")) {                      // ¿existe la constante?
    echo "El IVA es del " . IVA * 100 . " %";
}
```

	`const`	`define()`
Qué es	Una palabra del lenguaje	Una función
Cuándo se crea	Antes de ejecutar el programa	Al llegar a esa línea
Dentro de un `if`, un bucle o una función	❌ Error de sintaxis	✅ Sí
Nombre calculado (`"MONEDA_" . $idioma`)	❌ No	✅ Sí
Valor	Fijo o una operación con otras constantes	Cualquier valor, también el resultado de una función
Dentro de una clase (clase 17)	✅ Sí: `const IVA = 0.21;`	❌ No

En los dos casos, una vez creada, la constante **no se puede cambiar** ni volver a definir. Y para saber si existe, `defined("NOMBRE")`.



💡 **Mi consejo:** usad `const` siempre que podáis: se lee mejor, PHP la conoce desde el principio y es la que usaréis dentro de las clases. Dejad `define()` para cuando la necesitéis de verdad, por ejemplo para crear una constante solo si se cumple una condición.

📄 Ejemplo: `07-const-o-define.php`

✅ Dos pruebas para comprobarlo

Haced esto	Debéis obtener
Ejecutad `variables.php`	Me llamo Lucía y tengo 25 años.
Cambiad las comillas dobles por simples	Aparecen `$nombre` y `$edad` sin sustituir



Los operadores sirven para calcular, comparar y unir valores. El que más confunde al principio es el **punto (`.`)**, que en PHP une textos.

Aritméticos

Operador	Operación	Ejemplo	Resultado
`+`	Suma	`7 + 2`	9
`-`	Resta	`7 - 2`	5
`*`	Multiplicación	`7 * 2`	14
`/`	División	`7 / 2`	3.5
`%`	Resto (módulo)	`7 % 2`	1
`**`	Potencia	`7 ** 2`	49

Asignación y concatenación

08-operadores.php — php-paso-a-paso/clase04 — Visual Studio Code

🐘 08-operadores.php

```php
<?php
$x = 10;     // asigna
$x += 5;     // $x = $x + 5  → 15
$x++;        // suma 1       → 16

$saludo = "Hola, " . "Pedro";    // Hola, Pedro
$saludo .= "!";                  // añade al final → Hola, Pedro!
```

Comparación

Operador	Significado	Ejemplo	Resultado
`==`	Igual valor	`5 == "5"`	`true`
`===`	Igual valor **y** tipo	`5 === "5"`	`false`
`!==`	Distinto valor o tipo	`5 !== "5"`	`true`
`<` `>` `<=` `>=`	Menor, mayor...	`5 >= 5`	`true`

📄 Ejemplo: `08-operadores.php`

🗓️ Clase 5



CLASE 04 · PUNTO 7

🤔 Condicionales

Operadores lógicos

Operador	Significado	Es verdadero si...
`&&`	Y	Las dos condiciones son verdaderas
`||`	O	Al menos una es verdadera
`!`	NO	La condición es falsa

Tomar decisiones: if, elseif, else

Un bloque se puede escribir de **dos formas equivalentes**: con **llaves** `{ }` o con la **sintaxis alternativa**, que cambia la llave de apertura por `:` y la de cierre por `endif;`.

Con llaves

09-condicionales.php — php-paso-a-paso/clase04 — Visual Studio Code

09-condicionales.php

```php
<?php
$nota = 7.5;

if ($nota ⩾ 9) {
    echo "Sobresaliente";
} elseif ($nota ⩾ 7) {
    echo "Notable";
} elseif ($nota ⩾ 5) {
    echo "Aprobado";
} else {
    echo "Suspenso";
}
```

Sintaxis alternativa

09-condicionales.php — php-paso-a-paso/clase04 — Visual Studio Code

09-condicionales.php

```php
<?php
$nota = 7.5;

if ($nota ⩾ 9):
    echo "Sobresaliente";
elseif ($nota ⩾ 7):
    echo "Notable";
elseif ($nota ⩾ 5):
    echo "Aprobado";
else:
    echo "Suspenso";
endif;
```

Las condiciones se revisan de arriba abajo y solo se ejecuta el **primer** bloque que se cumple: con `$nota = 7.5` se muestra Notable.

Operador ternario: un if/else en una línea

09-condicionales.php — php-paso-a-paso/clase04 — Visual Studio Code

09-condicionales.php

```php
<?php
$edad = 16;
$mensaje = ($edad ⩾ 18) ? "Mayor de edad" : "Menor de edad";
```

📄 Ejemplo: `09-condicionales.php`



⚠️ **Fijaos:** `=` asigna y `===` compara. Escribir `if ($x = 5)` no compara: guarda un 5 en `$x` y la condición siempre se cumple.



CLASE 04 · PUNTO 8

👌 switch y match

Cuando comparáis **una variable con varios valores concretos**, `switch` y `match` quedan más claros que una cadena de `elseif`.

10-switch-match.php — php-paso-a-paso/clase04 — Visual Studio Code

10-switch-match.php

```php
<?php
$dia = "sábado";

switch ($dia) {
    case "sábado":
    case "domingo":
        echo "Fin de semana";
        break;
    default:
        echo "Día laborable";
}
```

`match` llegó con PHP 8 y devuelve directamente un valor:

10-switch-match.php — php-paso-a-paso/clase04 — Visual Studio Code

10-switch-match.php

```php
<?php
$codigo = 404;

$texto = match ($codigo) {
    200 => "Correcto",
    404 => "Página no encontrada",
    500 => "Error del servidor",
    default => "Código desconocido",
};
echo $texto;   // Página no encontrada
```

	`switch`	`match`
Compara con	`==` (solo valor)	`===` (valor y tipo)
Necesita `break`	Sí	No
Devuelve un valor	No	Sí
Sintaxis alternativa	Sí: `endswitch;`	No: solo llaves

📄 Ejemplo: `10-switch-match.php`



⚠️ **Fijaos:** sin `break`, `switch` sigue ejecutando los `case` de debajo.



CLASE 04 · PUNTO 9

🚀 Bucles

Bucle	Usadlo cuando…
`for`	Sabéis cuántas veces repetir
`while`	Repetís mientras se cumpla una condición
`do…while`	El bloque debe ejecutarse al menos una vez
`foreach`	Recorréis los elementos de un array

for: tabla de multiplicar

11-bucles.php — php-paso-a-paso/clase04 — Visual Studio Code

11-bucles.php

```php
<?php
for ($i = 1; $i ⩽ 10; $i++) {
    echo "5 x $i = ", 5 * $i, "<br>";
}
```

Sus tres partes: valor inicial (`$i = 1`), condición para seguir (`$i ⩽ 10`) y cambio en cada vuelta (`$i++`).

while: repetir mientras se cumpla

11-bucles.php — php-paso-a-paso/clase04 — Visual Studio Code

11-bucles.php

```php
<?php
$cuenta = 3;
while ($cuenta > 0) {
    echo "Quedan $cuenta<br>";
    $cuenta--;
}
```



🔁 **Fijaos:** si la condición nunca deja de cumplirse (por ejemplo, olvidáis `$cuenta--`), creáis un **bucle infinito** y la página se queda cargando.

📄 Ejemplo: `11-bucles.php`

foreach, do...while, break y continue

12-mas-bucles.php — php-paso-a-paso/clase04 — Visual Studio Code

12-mas-bucles.php

```php
<?php
$frutas = ["manzana", "pera", "uva"];
foreach ($frutas as $fruta) {
    echo "$fruta<br>";
}

$n = 10;
do {
    echo "Se ejecuta una vez aunque la condición sea falsa";
} while ($n < 5);

for ($i = 1; $i <= 10; $i++) {
    if ($i % 2 == 0) continue;    // salta los pares
    if ($i > 7) break;            // termina al pasar de 7
    echo "$i ";                   // 1 3 5 7
}
```

`break` sale del bucle al momento; `continue` salta a la siguiente vuelta. `do…while` es el único bucle que **no tiene sintaxis alternativa**.

📄 Ejemplo: `12-mas-bucles.php`

✅ Dos pruebas para comprobarlo

Haced esto	Debéis obtener
Ejecutad el `for` con llaves y luego con `endfor;`	La misma tabla del 5 en los dos casos
Quitad el `endforeach;` del ejemplo alternativo	*Parse error*: PHP no encuentra el cierre del bloque



CLASE 04 · PUNTO 10

🔍 Llaves o sintaxis alternativa

Todas las estructuras de bloque admiten las dos formas, salvo `do…while` y `match` [3].

Estructura	Con llaves	Sintaxis alternativa
`if`	`if (...) { } elseif else`	`if (...): elseif (...): else: endif;`
`switch`	`switch (...) { }`	`switch (...): … endswitch;`
`for`	`for (...) { }`	`for (...): … endfor;`
`while`	`while (...) { }`	`while (...): … endwhile;`
`foreach`	`foreach (...) { }`	`foreach (...): … endforeach;`
`do…while` , `match`	Solo llaves	No existe

¿Cuándo usar cada una?

- Para código PHP puro (cálculos, funciones, archivos que solo tienen PHP), las **llaves**.
- Para páginas que mezclan PHP y HTML, la **alternativa**: un `<?php endif; ?>` se distingue mucho mejor entre etiquetas que una `}` perdida.

13-sintaxis-alternativa-html.php — php-paso-a-paso/clase04 — Visual Studio Code

13-sintaxis-alternativa-html.php

```php
<?php if ($logueado): ?>
    <p>Bienvenido</p>
<?php else: ?>
    <a href="login.php">Entrar</a>
<?php endif; ?>
```

📄 Ejemplo: `13-sintaxis-alternativa-html.php`

Reglas que debéis recordar

- En cada estructura usad **una sola forma**: no mezcléis `{` con `endif;`.
- En la alternativa escribid `elseif` junto: `else if` da error.
- Cada cierre (`endif;` , `endfor;` ...) lleva su **punto y coma**, salvo que justo después venga `?>` .
- En un `switch` alternativo no pongáis HTML ni espacios entre `switch (...):` y el primer `case` .



Un array guarda varios valores en una sola variable. Los **indexados** usan posiciones numéricas que empiezan en 0; los **asociativos** usan claves con nombre.

14-arrays.php — php-paso-a-paso/clase04 — Visual Studio Code

14-arrays.php

```php
<?php
// Indexado
$colores = ["rojo", "verde", "azul"];
echo $colores[0];        // rojo (¡el primero es el 0!)
$colores[] = "negro";    // añade al final

// Asociativo
$alumno = ["nombre" => "Marta", "edad" => 22];
echo $alumno["nombre"];  // Marta

foreach ($alumno as $clave => $valor) {
    echo "$clave: $valor<br>";
}
```

Arrays dentro de arrays

14-arrays.php — php-paso-a-paso/clase04 — Visual Studio Code

14-arrays.php

```php
<?php
$alumnos = [
    ["nombre" => "Marta", "nota" => 8],
    ["nombre" => "Luis",  "nota" => 6],
];
foreach ($alumnos as $a) {
    echo $a["nombre"] . " tiene un " . $a["nota"] . "<br>";
}
```

Funciones útiles

Función	Qué hace	Ejemplo
`count()`	Cuenta elementos	`count($colores)` → 4
`in_array()`	¿Existe el valor?	`in_array("rojo", $colores)`
`sort()`	Ordena de menor a mayor	`sort($numeros)`
`implode()`	Une en un texto	`implode(", ", $colores)`
`print_r()`	Muestra todo el array	`print_r($alumno)`

📄 Ejemplo: `14-arrays.php`



CLASE 04 · PUNTO 12

👍 Funciones

Una función es un bloque de código con nombre: lo escribís una vez y lo usáis las veces que queráis. Recibe datos (**parámetros**) y devuelve un resultado con `return`.

15-funciones.php — php-paso-a-paso/clase04 — Visual Studio Code

15-funciones.php

```php
<?php
function saludar($nombre) {
    return "Hola, $nombre";
}

echo saludar("Carlos");    // Hola, Carlos
echo saludar("Elena");     // Hola, Elena
```

Parámetros con valor por defecto

15-funciones.php — php-paso-a-paso/clase04 — Visual Studio Code

15-funciones.php

```php
<?php
function calcularPrecio($base, $iva = 0.21) {
    return $base * (1 + $iva);
}

echo calcularPrecio(100);          // 121 (usa el 21 %)
echo calcularPrecio(100, 0.10);    // 110
```

Declarar tipos

Indicar el tipo de los parámetros y del resultado hace el código más claro, y PHP avisa si recibe un dato incorrecto.

15-funciones.php — php-paso-a-paso/clase04 — Visual Studio Code

15-funciones.php

```php
<?php
function sumar(int $a, int $b): int {
    return $a + $b;
}

function esMayorDeEdad(int $edad): bool {
    return $edad ≥ 18;
}
```

📄 Ejemplo: `15-funciones.php`



💡 El cuerpo de una función **siempre se escribe con llaves**. La sintaxis alternativa solo existe para `if`, `switch` y los bucles.

Ámbito: cada función tiene sus variables

Una función **no ve** las variables de fuera, y las que crea dentro desaparecen al terminar. Pasad siempre los datos como parámetros.

16-ambito-funciones-internas.php — php-paso-a-paso/clase04 — Visual Studio Code

16-ambito-funciones-internas.php

```php
<?php
$mensaje = "Hola";

function prueba() {
    echo $mensaje;     // Aviso: $mensaje no existe aquí
}

function pruebaBien($texto) {
    echo $texto;       // Correcto
}
pruebaBien($mensaje);
```

Funciones de PHP que usaréis a menudo

Función	Qué hace	Ejemplo
`strlen()`	Longitud de un texto	`strlen("Hola")` → 4
`strtoupper()`	Pasa a mayúsculas	`strtoupper("php")` → PHP
`trim()`	Quita espacios sobrantes	`trim(" hola ")` → hola
`str_replace()`	Reemplaza texto	`str_replace("a", "o", "casa")` → coso
`round()`	Redondea	`round(3.567, 2)` → 3.57
`rand()`	Número aleatorio	`rand(1, 6)`
`isset()`	¿Existe y no es `null`?	`isset($_POST["email"])`

📄 Ejemplo: `16-ambito-funciones-internas.php`

✅ Dos pruebas para comprobarlo

Haced esto	Debéis obtener
Llamad a `sumar(2, 3)`	5
Llamad a `prueba()`	Aviso «Undefined variable $mensaje»

🗓️ Clase 6



CLASE 04 · PUNTO 13

⚠️ Formularios: GET y POST

Aquí es donde la programación web cobra sentido: el usuario escribe algo, lo envía y nuestro PHP lo recibe. 🙌



PHP recibe los datos de un formulario en dos arrays especiales: `$_GET` y `$_POST` [4]. La clave de cada dato es el atributo `name` del campo.

Método	Dónde viajan los datos	Usadlo para
GET	En la URL: `buscar.php?q=php`	Búsquedas y filtros que se pueden compartir
POST	Ocultos en la petición	Registros, contraseñas y datos que cambian algo

El ejemplo más sencillo con GET

Empezamos con **dos archivos**: uno con el formulario y otro que recibe los datos. El atributo `action` dice a qué página se envían, y `method`, cómo:

17-formulario-get.html — php-paso-a-paso/clase04 — Visual Studio Code

17-formulario-get.html

```
<!-- 17-formulario-get.html: la página que PREGUNTA -->
<form action="18-recibir-get.php" method="get">
    Nombre: <input type="text" name="nombre">
    Ciudad: <input type="text" name="ciudad">
    <button type="submit">Enviar</button>
</form>
```

17-formulario-get.html — php-paso-a-paso/clase04 — Visual Studio Code

17-formulario-get.html

```
<?php
// 18-recibir-get.php: la página que RESPONDE
echo "Hola, " . $_GET["nombre"] . "<br>";
echo "Vives en " . $_GET["ciudad"];
```

Escribid vuestro nombre y vuestra ciudad, pulsad **Enviar** y mirad la barra de direcciones:

Esquema

```
http://php-basico.test/18-recibir-get.php?nombre=Ana&ciudad=Santander
```

Después del `?` van los datos: `nombre=Ana` y `ciudad=Santander`, separados por `&`. Cada `name` del formulario se convierte en una clave de `$_GET`. Como están en la URL, podéis cambiarlos a mano, guardar la dirección en favoritos o enviársela a alguien.

📄 Ejemplos: `17-formulario-get.html` y `18-recibir-get.php`

El mismo ejemplo con POST

Solo cambian dos cosas: `method="post"` en el formulario y `$_POST` en lugar de `$_GET` en la página que recibe.

19-formulario-post.html — php-paso-a-paso/clase04 — Visual Studio Code

19-formulario-post.html

```html
<!-- 19-formulario-post.html -->
<form action="20-recibir-post.php" method="post">
    Usuario: <input type="text" name="usuario">
    Contraseña: <input type="password" name="clave">
    <button type="submit">Entrar</button>
</form>
```

19-formulario-post.html — php-paso-a-paso/clase04 — Visual Studio Code

19-formulario-post.html

```php
<?php
// 20-recibir-post.php
echo "Usuario: " . $_POST["usuario"] . "<br>";
echo "La contraseña tiene " . strlen($_POST["clave"]) . " caracteres";
```

Ahora la URL es solo `20-recibir-post.php`: los datos viajan **dentro** de la petición. ¿Os imagináis la contraseña escrita en la barra de direcciones (y en el historial) con GET? Por eso los formularios de acceso, de registro o los que cambian algo van siempre por POST.



🔍 Para verlos, abrid las herramientas del navegador (**F12**), pestaña **Red**, pulsad sobre la petición y mirad la sección **Carga** (*Payload*). POST oculta los datos de la URL, pero **no los cifra**: eso lo hace HTTPS.

📄 Ejemplos: `19-formulario-post.html` y `20-recibir-post.php`

Dos problemas: los espacios y el HTML que escribe el usuario

Estos dos ejemplos funcionan, pero son muy confiados. Probad estas tres cosas:

1. Abrid `18-recibir-get.php` directamente, sin pasar por el formulario: PHP avisa con un *Warning: Undefined array key "nombre"*, porque esa clave no existe.
2. Escribid `Ana` con espacios delante y detrás: llegan tal cual.
3. Escribid `<b>Ana</b>` como nombre: ¡sale en negrita! El navegador ha interpretado vuestro texto como HTML. Si en lugar de `<b>` alguien escribe un `<script>`, se ejecutaría: es un ataque **XSS**.

Tres herramientas lo arreglan, y a partir de ahora las usaremos siempre:

Herramienta	Qué hace
`$_POST["nombre"] ?? ""`	Si la clave no existe, usa un texto vacío y no hay aviso
`trim()`	Quita los espacios del principio y del final
`htmlspecialchars()`	Convierte `<` en `<`, `>` en `>`...: el texto se ve, pero no se ejecuta [5]

21-formulario-saludo.html — php-paso-a-paso/clase04 — Visual Studio Code

21-formulario-saludo.html

```html
<!-- 21-formulario-saludo.html -->
<form action="22-recibir-saludo.php" method="post">
    Nombre: <input type="text" name="nombre">
    <button type="submit">Saludar</button>
</form>
```

21-formulario-saludo.html — php-paso-a-paso/clase04 — Visual Studio Code

21-formulario-saludo.html

```php
<?php
// 22-recibir-saludo.php
$nombre = trim($_POST["nombre"] ?? "");
?>
<?php if ($nombre === ""): ?>
    <p>No has escrito tu nombre. <a href="21-formulario-saludo.html">Volver</a></p>
<?php else: ?>
    <p>¡Hola, <?php echo htmlspecialchars($nombre); ?>! 👋</p>
<?php endif; ?>
```

📄 Ejemplos: `21-formulario-saludo.html` y `22-recibir-saludo.php`

Fijaos en que, al enviar, **cambiáis de página**. Si hay un error, el usuario tiene que volver atrás y puede perder lo que había escrito. Por eso, en los ejemplos siguientes, el formulario se envía **a sí mismo**: sin `action`, la misma página pregunta y responde. En la clase 12 compararemos con calma las dos formas y veremos cómo separar «lo que decide» de «lo que se ve», la idea que acabará siendo el MVC.

¿Se ha enviado el formulario? Con `REQUEST_METHOD`

Cuando la misma página muestra el formulario y lo procesa, necesita saber en qué momento está: ¿acabamos de llegar o nos han enviado datos? `$_SERVER["REQUEST_METHOD"]` dice con qué método se ha pedido la página. La primera vez que entramos es `GET` (escribir una dirección o pulsar un enlace siempre es GET) y, al pulsar el botón de un formulario POST, es `POST`:

23-enviado-post.php — php-paso-a-paso/clase04 — Visual Studio Code

23-enviado-post.php

```php
<?php
$mensaje = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nombre = trim($_POST["nombre"] ?? "");
    $mensaje = "Formulario recibido. Hola, " . htmlspecialchars($nombre) . ".";
}
?>
<?php if ($mensaje !== ""): ?>
    <p><?php echo $mensaje; ?></p>
<?php else: ?>
    <p>Todavía no has enviado el formulario.</p>
<?php endif; ?>

<form method="post">
    Nombre: <input type="text" name="nombre">
    <button type="submit">Enviar</button>
</form>
```

📄 Ejemplo: `23-enviado-post.php`

¿Y con GET? Aquí está la trampa: **con GET no sirve**. Al entrar en la página, el método es `GET`, y al enviar el formulario… también es `GET`. Lo que cambia es que, al enviar, `$_GET` trae los campos. Así que preguntamos si existe uno con `isset()`:

24-enviado-get.php — php-paso-a-paso/clase04 — Visual Studio Code

24-enviado-get.php

```php
<?php
$mensaje = "";

// Al entrar, $_GET está vacío; al enviar, trae "nombre" (aunque esté en blanco)
if (isset($_GET["nombre"])) {
    $nombre = trim($_GET["nombre"]);
    $mensaje = "Formulario recibido. Hola, " . htmlspecialchars($nombre) . ".";
}

$metodo = $_SERVER["REQUEST_METHOD"];   // "GET" al entrar y "GET" al enviar
?>
<p>Método de esta petición: <strong><?php echo $metodo; ?></strong></p>
<!-- ...el mismo mensaje y el mismo formulario, con method="get" -->
```

📄 Ejemplo: `24-enviado-get.php`

¿Se ha enviado el formulario? Con el `name` del botón

Hay otra forma muy habitual, y funciona igual con los dos métodos: ponerle un `name` **al botón**. El botón pulsado también se envía con el formulario, así que su clave solo existe en `$_POST` (o en `$_GET`) cuando alguien lo ha pulsado:

25-boton-post.php — php-paso-a-paso/clase04 — Visual Studio Code

25-boton-post.php

```php
<?php
$mensaje = "";

if (isset($_POST["enviar"])) {     // ¿se ha pulsado el botón "enviar"?
    $nombre = trim($_POST["nombre"] ?? "");
    $mensaje = "Has pulsado «Enviar». Hola, " . htmlspecialchars($nombre) . ".";
}
?>
<?php if ($mensaje !== ""): ?>
    <p><?php echo $mensaje; ?></p>
<?php endif; ?>

<form method="post">
    Nombre: <input type="text" name="nombre">
    <button type="submit" name="enviar">Enviar</button>
</form>
```

📄 Ejemplo: `25-boton-post.php`

Con GET es igual, cambiando `$_POST` por `$_GET` y `method="post"` por `method="get"`. Al enviar, veréis el botón en la URL: `26-boton-get.php?nombre=Ana&enviar=`.

📄 Ejemplo: `26-boton-get.php`

Forma de comprobarlo	Con POST	Con GET
`$_SERVER["REQUEST_METHOD"] === "POST"`	✅	❌ Siempre es GET
`isset($_GET["campo"])` / `isset($_POST["campo"])`	✅	✅
`isset($_POST["boton"])` con `name` en el botón	✅	✅

Un formulario con varios botones

La ventaja del `name` en el botón se ve cuando hay **varios**: solo llega el que se ha pulsado, así que sabemos qué quería hacer el usuario.

27-calculadora-dos-botones.php — php-paso-a-paso/clase04 — Visual Studio Code

27-calculadora-dos-botones.php

```php
<?php
$numero1 = (float) ($_POST["numero1"] ?? 0);
$numero2 = (float) ($_POST["numero2"] ?? 0);
$resultado = "";

if (isset($_POST["sumar"])) {
    $resultado = "$numero1 + $numero2 = " . ($numero1 + $numero2);
} elseif (isset($_POST["restar"])) {
    $resultado = "$numero1 - $numero2 = " . ($numero1 - $numero2);
} elseif (isset($_POST["multiplicar"])) {
    $resultado = "$numero1 × $numero2 = " . ($numero1 * $numero2);
}
?>
<form method="post">
    <input type="number" step="any" name="numero1" value="<?php echo $numero1; ?>">
    <input type="number" step="any" name="numero2" value="<?php echo $numero2; ?>">
    <button type="submit" name="sumar">Sumar</button>
    <button type="submit" name="restar">Restar</button>
    <button type="submit" name="multiplicar">Multiplicar</button>
</form>
```

Todo lo que llega de un formulario es **texto**, aunque el campo sea `type="number"`: por eso convertimos con `(float)` antes de operar. Y fijaos en el `value` de los campos: así los números siguen ahí después de calcular.

📄 Ejemplo: `27-calculadora-dos-botones.php`

Un buscador con GET que recuerda lo escrito

Un buscador sencillo que se envía a sí mismo. Aquí no hace falta `isset()`: con `?? ""`, al entrar `$q` vale un texto vacío y no mostramos nada. Fijaos en el `value` de la caja: después de buscar, lo que habíais escrito sigue ahí.

28-buscar-get.php — php-paso-a-paso/clase04 — Visual Studio Code

28-buscar-get.php

```php
<?php
$q = trim($_GET["q"] ?? "");
?>
<form method="get">
    <input type="text" name="q" value="<?php echo htmlspecialchars($q); ?>">
    <button type="submit">Buscar</button>
</form>

<?php if ($q !== ""): ?>
    <p>Has buscado: <strong><?php echo htmlspecialchars($q); ?></strong></p>
<?php endif; ?>
```

📄 Ejemplo: `28-buscar-get.php`

Otros tipos de campos

Hasta ahora solo hemos usado cajas de texto, pero un formulario tiene muchos más campos. Cada uno llega a PHP de una forma:

Campo	Qué llega	Si no se marca
`<input type="radio" name="nivel" value="...">`	El `value` del elegido	No llega nada
`<input type="checkbox" name="temas[]" value="...">`	Un **array** con los `value` marcados (por los `[]` del `name`)	No llega nada
`<select name="horario">`	El `value` de la opción elegida	Siempre llega una
`<textarea name="comentario">`	El texto, con sus saltos de línea	Llega vacío

29-encuesta.php — php-paso-a-paso/clase04 — Visual Studio Code

29-encuesta.php

```php
<?php
$enviado = isset($_POST["enviar"]);

$nivel = $_POST["nivel"] ?? "";              // radio
$temas = $_POST["temas"] ?? [];              // casillas: un array
$horario = $_POST["horario"] ?? "mañana";    // desplegable
$comentario = trim($_POST["comentario"] ?? "");
?>
<?php if ($enviado): ?>
    <p>Temas: <?php echo count($temas) > 0 ? htmlspecialchars(implode(", ", $temas)) :
"ninguno"; ?></p>
    <p>Comentario: <?php echo nl2br(htmlspecialchars($comentario)); ?></p>
<?php endif; ?>
```

`implode()` une los elementos de un array con el texto que le digáis, y `nl2br()` convierte los saltos de línea del `textarea` en `<br>`. Fijaos en el orden: **primero** `htmlspecialchars()` y **después** `nl2br()`; al revés, el `<br>` también se escaparía y se vería como texto.

📄 Ejemplo: `29-encuesta.php`

Un formulario POST con validación

🎯 **Objetivo:** crear `contacto.php`, un formulario que se envía a sí mismo, valida los datos y saluda al usuario. Como mezcla PHP con HTML, usa la **sintaxis alternativa**.

30-contacto.php — php-paso-a-paso/clase04 — Visual Studio Code

30-contacto.php

```php
<?php
$errores = [];
$nombre = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nombre = trim($_POST["nombre"] ?? "");
    $email  = trim($_POST["email"] ?? "");

    if ($nombre === "") {
        $errores[] = "El nombre es obligatorio.";
    }
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errores[] = "El email no es válido.";
    }
}
?>
<?php if ($_SERVER["REQUEST_METHOD"] === "POST" && empty($errores)): ?>
    <p>Gracias, <?php echo htmlspecialchars($nombre); ?>.</p>
<?php else: ?>
    <?php foreach ($errores as $error): ?>
        <p style="color:red"><?php echo $error; ?></p>
    <?php endforeach; ?>
    <form method="post">
        Nombre: <input type="text" name="nombre"
                       value="<?php echo htmlspecialchars($nombre); ?>">
        Email: <input type="email" name="email">
        <button type="submit">Enviar</button>
    </form>
<?php endif; ?>
```

📄 Ejemplo: `30-contacto.php`

¿Qué hace cada parte?

Código	Para qué sirve
`$_SERVER["REQUEST_METHOD"] === "POST"`	Saber si el formulario se ha enviado
`$_POST["nombre"] ?? ""`	Leer el dato o, si no llega, usar texto vacío
`trim()`	Quitar espacios al principio y al final
`filter_var(..., FILTER_VALIDATE_EMAIL)`	Comprobar que el email tiene formato válido
`htmlspecialchars()`	Mostrar el texto del usuario de forma segura [5]

🛡️ La regla de oro: no os fiéis del usuario

- **Validad siempre en el servidor.** Las comprobaciones de HTML (`required`, `type="email"`) se pueden saltar fácilmente.
- **Escapad al mostrar.** Si alguien escribe `<script>` en su nombre y lo mostráis sin `htmlspecialchars()`, el navegador lo ejecuta. Es un ataque **XSS**.
- **Volved a rellenar los campos** con lo que escribió el usuario cuando hay errores, para que no tenga que repetirlo.

✅ Pruebas para comprobarlo

Haced esto	Debéis obtener
Enviad `17-formulario-get.html` y luego cambiad el nombre en la URL	El saludo cambia sin volver a usar el formulario
Abrid `24-enviado-get.php` y enviadlo	El método es `GET` las dos veces, pero solo la segunda dice «Formulario recibido»
Pulsad «Restar» en la calculadora con 7 y 2,5	7 - 2.5 = 4.5, y los dos números siguen en sus cajas
Enviad `30-contacto.php` vacío	Los dos mensajes de error en rojo
Escribid `<b>Ana</b>` como nombre y un email válido	Gracias, `<b>Ana</b>`. (con las etiquetas visibles, no en negrita)



⚠️ **Fijaos:** el atributo `name` del input y la clave de `$_POST` deben coincidir exactamente, mayúsculas incluidas.



Los errores al empezar casi siempre son los mismos. Antes de pedir ayuda, **leed el mensaje**: os dice **qué** falla y **en qué línea**.

Lo que veis	Qué debéis revisar
`Parse error: syntax error`	Falta un `;`, una llave, un `endif;` o unas comillas. Mirad la línea indicada y la anterior
`Warning: Undefined variable`	La variable está mal escrita (¿mayúsculas?) o no se ha creado antes
`Warning: Undefined array key`	El campo no llegó en `$_POST`. Usad `?? ""` o `isset()`
Veis el código PHP en el navegador	Habéis abierto el archivo con doble clic. Usad `http://nombre.test/`
`nombre.test` no carga	Pulsad **Recargar** en Laragon. Si sigue, abridlo como administrador
Página en blanco	Hay un error y los avisos están ocultos (mirad abajo)

Ver todos los errores mientras desarrolláis

Laragon muestra los errores por defecto. Si veis una página en blanco, activadlos en el propio archivo:

31-mostrar-errores.php — php-paso-a-paso/clase04 — Visual Studio Code

31-mostrar-errores.php

```php
<?php
ini_set("display_errors", 1);
error_reporting(E_ALL);
```

📄 Ejemplo: `31-mostrar-errores.php`

O de forma permanente: clic derecho en Laragon → **PHP → php.ini**, poned `display_errors = On`, guardad y pulsad **Recargar**.



🔐 **Fijaos:** mostrar errores es solo para desarrollar. En un servidor real se desactivan, porque dan pistas a posibles atacantes.



CLASE 04 · PUNTO 15

😮 Conectar con MySQL

Último paso de este bloque: guardar y leer datos de verdad. PHP se comunica con MySQL mediante la extensión **PDO** (*PHP Data Objects*) [6]. Funciona igual con distintas bases de datos y permite **consultas preparadas**, que protegen vuestros datos. En el curso usaremos solo PDO.

Esquema

```
Script PHP  ──1──►  PDO  ──2──►  MySQL
(pide datos         (abre la      (Laragon,
 con SQL)            conexión)     puerto 3306)
      ◄──── 3 · MySQL devuelve las filas y PDO se las entrega a PHP como arrays
```



📌 Esto es solo una primera toma de contacto. Más adelante dedicaremos un bloque entero a SQL y otro a PDO.

Preparar la base de datos

🎯 **Objetivo:** crear la base de datos `academia` con una tabla `alumnos`.

1. En Laragon pulsad **Iniciar todo** y comprobad que MySQL está en marcha.
2. Pulsad **Base de datos** para abrir HeidiSQL y conectad (usuario `root`, sin contraseña).
3. Abrid una pestaña **Consulta**, pegad el SQL de abajo y ejecutadlo con **F9**.
4. Pulsad **F5** en el panel izquierdo: debe aparecer `academia` con la tabla `alumnos`.

consulta.sql — php-paso-a-paso/clase04 — Visual Studio Code

consulta.sql

```sql
CREATE DATABASE academia
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE academia;

CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    email VARCHAR(120) NOT NULL,
    nota DECIMAL(4,2)
);

INSERT INTO alumnos (nombre, email, nota) VALUES
    ('Marta', 'marta@ejemplo.com', 8.50),
    ('Luis', 'luis@ejemplo.com', 6.00);
```

📄 Ejemplo: `bd/academia.sql`

Los datos de la conexión

Dato	Valor en Laragon
Servidor (host)	`localhost`
Puerto	`3306` (por defecto)
Usuario / contraseña	`root` / vacía
Base de datos	`academia`
Juego de caracteres	`utf8mb4`

Paso 1: la conexión más sencilla

Empezamos con lo mínimo: conectar y leer la tabla, en un solo archivo y sin nada más.

conexion-sencilla.php — php-paso-a-paso/clase04 — Visual Studio Code

conexion-sencilla.php

```php
<?php
// Conectar: tipo de base de datos, servidor, nombre de la base y caracteres; usuario;
contraseña
$pdo = new PDO("mysql:host=localhost;dbname=academia;charset=utf8mb4", "root", "");

echo "<p>¡Conectados con MySQL!</p>";

// Pedir datos con una consulta SQL fija
$consulta = $pdo->query("SELECT nombre, email, nota FROM alumnos");

// Recorrer las filas: cada una es un array con los datos de un alumno
foreach ($consulta as $fila) {
    echo $fila["nombre"] . " (" . $fila["email"] . "): " . $fila["nota"] . "<br>";
}
```

📄 Ejemplo: `bd/conexion-sencilla.php`

Pieza	Qué hace
`new PDO(...)`	Abre la conexión. Desde ahora, `$pdo` es nuestra «línea directa» con MySQL
`"mysql:host=...;dbname=...;charset=..."`	El **DSN**: a qué base de datos nos conectamos y cómo
`$pdo->query("SELECT ...")`	Envía la consulta a MySQL y nos devuelve el resultado
`foreach ($consulta as $fila)`	Recorre las filas una a una, como un array

Ahora cambiad `dbname=academia` por `dbname=academiaa` y recargad: aparece un **error fatal** (*Fatal error: Uncaught PDOException... Unknown database*) con un montón de líneas que el usuario no debería ver, y

el programa se detiene ahí.

Paso 2: controlar el error con try/catch

Con `try` / `catch` [10] le decimos a PHP: «intenta hacer esto (`try`) y, si algo falla, en lugar de detenerte con un error, haz esto otro (`catch`)». En la clase 14 lo veremos a fondo; de momento, basta con esta idea:

conexion-try-catch.php — php-paso-a-paso/clase04 — Visual Studio Code

conexion-try-catch.php

```php
<?php
try {
    $pdo = new PDO("mysql:host=localhost;dbname=academia;charset=utf8mb4", "root", "");

    echo "<p>¡Conectados con MySQL!</p>";

    $consulta = $pdo->query("SELECT nombre, nota FROM alumnos");

    foreach ($consulta as $fila) {
        echo $fila["nombre"] . ": " . $fila["nota"] . "<br>";
    }
} catch (PDOException $e) {
    // Si la conexión o la consulta fallan, llegamos aquí
    echo "<p>No se ha podido conectar con la base de datos.</p>";
    echo "<p>Motivo: " . $e->getMessage() . "</p>";
}
```

📄 Ejemplo: `bd/conexion-try-catch.php`

Repetid la prueba con `dbname=academiaa`: ahora sale nuestro mensaje y el motivo (*Unknown database 'academiaa'*). `$e` es la **excepción**, un objeto con la información del error, y `getMessage()` nos da su descripción.



🔐 Mostrar `getMessage()` está bien mientras aprendéis, pero en una web publicada no: puede dar pistas sobre vuestro servidor. Allí se enseña un mensaje genérico y el detalle se guarda en un registro.

Paso 3: la conexión completa, en su propio archivo

En un proyecto real, la conexión se escribe **una sola vez** en un archivo, `conexion.php`, que incluyen todos los demás. Y le añadimos dos opciones que nos harán la vida más fácil:

conexion.php — php-paso-a-paso/clase04 — Visual Studio Code

conexion.php

```php
<?php
$dsn = "mysql:host=localhost;dbname=academia;charset=utf8mb4";

try {
    $pdo = new PDO($dsn, "root", "", [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);
} catch (PDOException $e) {
    die("Error de conexión: " . $e->getMessage());
}
```

📄 Ejemplo: `bd/conexion.php`

Pieza	Qué hace
`$dsn`	El mismo texto de antes, guardado en una variable para que se lea mejor
`new PDO(...)`	Abre la conexión: recibe el DSN, el usuario, la contraseña y un array de **opciones**
`ERRMODE_EXCEPTION`	Si algo falla, PHP lanza una excepción que recogemos con `catch` (en PHP 8 ya es así por defecto, pero lo dejamos escrito para que se vea)
`FETCH_ASSOC`	Cada fila llega como array asociativo: `$fila["nombre"]`
`die()`	Muestra el mensaje y detiene el programa: sin conexión, no tiene sentido seguir



🔐 `root` sin contraseña solo vale en vuestro equipo. En un servidor real se crea un usuario propio con los permisos justos.

Probar la conexión

probar.php — php-paso-a-paso/clase04 — Visual Studio Code

probar.php

```php
<?php
require "conexion.php";
echo "Conexión correcta con MySQL " . $pdo->getAttribute(PDO::ATTR_SERVER_VERSION);
```

`require` incluye otro archivo PHP: así `conexion.php` se escribe una vez y se reutiliza en todos los scripts.

📄 Ejemplo: `bd/probar.php`

Leer datos

Con `conexion.php` listo, cada script empieza con un `require` y se centra en lo suyo. PDO ofrece dos caminos: `query()`, el que hemos usado hasta ahora, para consultas fijas, y `prepare()` con `execute()` cuando en la consulta intervienen valores variables.

leer.php — php-paso-a-paso/clase04 — Visual Studio Code

leer.php

```php
<?php
require "conexion.php";

$consulta = $pdo->query("SELECT nombre, nota FROM alumnos");

foreach ($consulta as $fila) {
    echo $fila["nombre"] . ": " . $fila["nota"] . "<br>";
}
```

📄 Ejemplo: `bd/leer.php`

Método	Devuelve	Usadlo para
`fetch()`	Una fila, o `false` si no hay más	Un solo registro, por ejemplo por id
`fetchAll()`	Todas las filas en un array	Listados completos
`fetchColumn()`	El valor de la primera columna	Contar: `SELECT COUNT(*) ...`

Con **consultas preparadas**, la consulta lleva **marcadores** (`:minima`) y los valores se envían aparte con `execute()`:

aprobados.php — php-paso-a-paso/clase04 — Visual Studio Code

aprobados.php

```php
<?php
require "conexion.php";

$stmt = $pdo->prepare("SELECT nombre, nota FROM alumnos WHERE nota >= :minima");
$stmt->execute(["minima" => 7]);

foreach ($stmt->fetchAll() as $a) {
    echo $a["nombre"] . " (" . $a["nota"] . ")<br>";
}
```

📄 Ejemplo: `bd/aprobados.php`

Escribir datos



🚨 **Nunca peguéis datos del usuario dentro de la consulta.**

```php // MAL: inyección SQL $sql = "SELECT * FROM alumnos WHERE id = " . $_GET["id"];

// BIEN: consulta preparada $stmt = $pdo->prepare("SELECT * FROM alumnos WHERE id = :id"); $stmt->execute(["id" => $_GET["id"]]); ```

Si alguien escribe `?id=1 OR 1=1`, la primera consulta devuelve **todos** los alumnos. Con marcadores, PDO trata el valor siempre como un dato y nunca como parte del SQL [7].

cambiar.php — php-paso-a-paso/clase04 — Visual Studio Code

cambiar.php

```php
<?php
require "conexion.php";

// INSERT: añadir un alumno
$stmt = $pdo->prepare("INSERT INTO alumnos (nombre, email, nota)
                       VALUES (:nombre, :email, :nota)");
$stmt->execute(["nombre" => "Ana", "email" => "ana@ejemplo.com", "nota" => 9]);
echo "Nuevo id: " . $pdo->lastInsertId();

// UPDATE: cambiar una nota
$stmt = $pdo->prepare("UPDATE alumnos SET nota = :nota WHERE id = :id");
$stmt->execute(["nota" => 7.5, "id" => 2]);
echo $stmt->rowCount() . " fila(s) modificada(s)";

// DELETE: borrar un alumno
$stmt = $pdo->prepare("DELETE FROM alumnos WHERE id = :id");
$stmt->execute(["id" => 3]);
```

📄 Ejemplo: `bd/cambiar.php`

Método	Para qué sirve
`lastInsertId()`	Devuelve el id que MySQL asignó al último `INSERT`
`rowCount()`	Cuenta las filas afectadas por `INSERT`, `UPDATE` o `DELETE`



⚠️ **Fijaos:** sin `WHERE`, un `UPDATE` o un `DELETE` afecta a **todas** las filas de la tabla. Revisad siempre el `WHERE` antes de ejecutar.

Ejemplo completo: alta y listado de alumnos

Juntamos todo lo aprendido: un formulario que **añade alumnos** a la base de datos y **lista** los que ya existen. Usa la sintaxis alternativa y `<?=`, el atajo de `<?php echo`.
```

📄 Ejemplo: `bd/alumnos.php`

✅ Pruebas para comprobarlo

Haced esto	Debéis obtener
Abrid `conexion-sencilla.php`	¡Conectados con MySQL! y la lista de alumnos
Cambiad en `conexion-try-catch.php` el nombre de la base por `academiaa`	Vuestro mensaje con el motivo: *Unknown database*
Abrid `probar.php`	Conexión correcta y la versión de MySQL
Enviad en `alumnos.php` un nombre y un email válidos	El alumno aparece en la lista y en HeidiSQL (F5)
Enviad el formulario vacío	Dos mensajes de error y no se inserta nada

Si algo falla, empezad por aquí

Lo que veis	Qué debéis revisar
`could not find driver`	Extensión inactiva: clic derecho en Laragon → **PHP → Extensiones** → `pdo_mysql`
`SQLSTATE[HY000] [2002]`	MySQL no está arrancado o usa otro puerto. Pulsad **Iniciar todo**
`[1045] Access denied`	Usuario o contraseña incorrectos. En Laragon: `root` sin contraseña
`[1049] Unknown database`	La base no existe o el nombre está mal escrito. Revisadlo en HeidiSQL
Caracteres raros (`Ã±` , `Ã©`)	Falta `charset=utf8mb4` en el DSN



CLASE 04 · PUNTO 16

🚀 Ejercicios propuestos

Cread un archivo por ejercicio dentro de `C:\laragon\www\php-basico` y probadlo en `http://php-basico.test/`. Marcad cada uno al terminarlo. Tenéis una solución de cada uno en `soluciones/` (las de la base de datos, en `soluciones/bd/`, junto a su `conexion.php`): miradla solo después de intentarlo.

- ☐ **01 · Presentación.** Guardad vuestro nombre, edad y ciudad en variables y mostradlos en una frase con `echo`. → `ej01-presentacion.php`
- ☐ **02 · Calculadora de IVA.** Con una constante `IVA`, calculad el precio final de un producto de 59,90 €. → `ej02-iva.php`
- ☐ **03 · Par o impar.** Dado un número, indicad si es par o impar con el operador `%`. → `ej03-par-impar.php`

- ☐ **04 · Calificación.** Convertid una nota en Suspenso, Aprobado, Notable o Sobresaliente: primero con `if` y llaves, luego con la sintaxis alternativa y por último con `match`. → `ej04-calificacion.php`
- ☐ **05 · Tabla de multiplicar.** Generad con `for` una tabla HTML con la tabla del número que elijáis. → `ej05-tabla-multiplicar.php`
- ☐ **06 · Lista de la compra.** Recorred un array de productos con `foreach` y `endforeach;` y mostradlos en una lista `<ul>`. → `ej06-lista-compra.php`
- ☐ **07 · Media de notas.** Cread `calcularMedia(array $notas): float` y usadla con un array de alumnos. → `ej07-media-notas.php`
- ☐ **08 · Registro.** Pedid nombre, email y edad por POST, validad los tres campos y mostrad los errores o un mensaje de bienvenida. → `ej08-registro.php`
- ☐ **09 · Listado.** Mostrad todos los alumnos de la tabla `alumnos` en una tabla HTML con `foreach`. → `ej09-listado.php`
- ☐ **10 · Buscador.** Cread un formulario GET que busque alumnos por nombre con `LIKE` y una consulta preparada. → `ej10-buscador.php`
- ☐ **11 · Gestión.** Añadid a `alumnos.php` un botón «Borrar» por alumno que envíe el id por POST y ejecute un `DELETE`. → `ej11-gestion.php`



🏆 **Vuestro reto final:** unid los ejercicios 04 y 08 en un formulario que pida un nombre y una nota, muestre la calificación y guarde el resultado en la base de datos. Si sabéis explicar qué hace cada línea, lo habéis entendido.



CLASE 04 · PUNTO 17

👋 Glosario

Término	Definición
Servidor web	Programa que recibe peticiones y devuelve páginas. En Laragon, Apache
Intérprete	Programa que lee y ejecuta el código PHP
Host virtual	Dirección propia para un proyecto, como `php-basico.test`
phpinfo()	Función que muestra la configuración y las extensiones de PHP en el servidor
Variable	Espacio con nombre que guarda un dato; empieza por `$`
Constante	Valor con nombre que no cambia durante la ejecución
`const` / `define()`	Las dos formas de crear una constante: `const` va en el nivel principal del código; `define()` es una función y puede ir dentro de un `if`
Sintaxis alternativa	Forma de escribir `if` , `for` , `while` … con `:` y `endif;` , `endfor;` … en lugar de llaves
Array	Varios valores guardados en una misma variable
Función	Bloque de código reutilizable con nombre
Parámetro	Dato que recibe una función
Superglobal	Variable disponible en todo el script: `$_GET` , `$_POST` , `$_SERVER`
`isset()`	Pregunta si una variable o una clave de un array existe (y no es `null`)
`REQUEST_METHOD`	Clave de `$_SERVER` con el método de la petición: `GET` o `POST`
XSS	Ataque que inyecta código en una página a través de datos del usuario
PDO	Extensión de PHP para conectar con bases de datos como MySQL
DSN	Cadena que indica a PDO qué base de datos, servidor y caracteres usar
try / catch	Bloque que intenta ejecutar un código y, si lanza una excepción, la recoge en lugar de detener el programa
Consulta preparada	Consulta SQL con marcadores (`:id`) a los que se envían los valores aparte
Inyección SQL	Ataque que altera una consulta a través de datos del usuario mal tratados

📚 Fuentes y documentación oficial

1. Manual de PHP: ¿Qué es PHP?
2. Manual de PHP: phpinfo
3. Manual de PHP: sintaxis alternativa de estructuras de control
4. Manual de PHP: variables desde fuentes externas (formularios)
5. Manual de PHP: htmlspecialchars
6. Manual de PHP: PDO (PHP Data Objects)
7. Manual de PHP: sentencias preparadas y procedimientos almacenados
8. Manual de PHP: etiquetas de PHP
9. Manual de PHP: sintaxis de las constantes (`const` y `define()`)
10. Manual de PHP: excepciones (`try` y `catch`)